

Домашнее задание 5. Обеспечение воспроизводимости эксперимента

1. Выяснение ограничений традиционных ноутбуков Collab

1. Создаем репозиторий с именем hw5_mlops_Rtishcheva_Alena с помощью GitHub Desktop.
2. В папке hw5_mlops_Rtishcheva_Alena создаем виртуальное окружение
`python -m venv venv`
3. Активируем виртуальное окружение
`venv\Scripts\activate`
4. Устанавливаем библиотеки
`pip install pandas scikit-learn dvc mlflow pyyaml marimo feast`
5. Создаем файл `marimo_test.py`

```
marimo_test.py > ...
1  import marimo
2
3  __generated_with__ = "0.23.3"
4  app = marimo.App()
5
6
7  @app.cell
8  def _():
9      import marimo as mo
10     import pandas as pd
11
12     return mo, pd
13
14
15  @app.cell
16  def _(mo):
17      mo.md("""
18      # Задание 1: Сравнение Marimo и Colab
19      """)
20      return
21
22
23  @app.cell
24  def _(pd):
25      # Создаем DataFrame
26      df = pd.DataFrame({
27          "Пассажир": ["Алена", "Иван", "Мария"],
28          "Въехал": [1, 0, 1],
29          "Класс": [1, 3, 2]
30      })
31      return (df,)
32
33
34  @app.cell
35  def _(df):
36      # Отображаем таблицу
37      df
38      return
39
40
41  if __name__ == "__main__":
42      app.run()
```

Рисунок 1 – Создание файла `marimo_test.py`

6. Запускаем в терминале

```
PS D:\hw5_mlops_Rtishcheva_Alena> marimo edit marimo_test.py

Edit marimo_test.py in your browser 📄

→ URL: http://localhost:2718?access_token=1lkR3V-PDqY8L6aIKeVQZQ

💡 Tip: Lint and format notebooks
$ marimo check --fix .
```

Рисунок 2 – Запуск в терминале

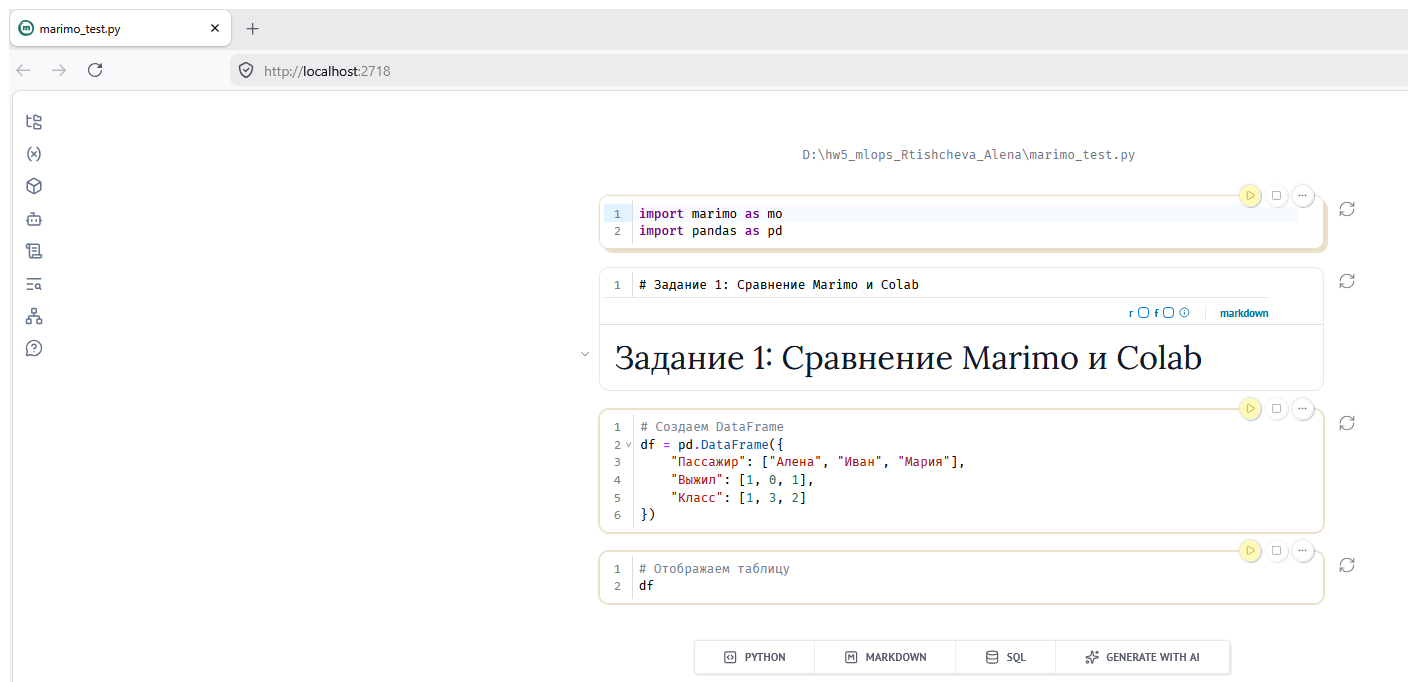


Рисунок 3 – Открытие в браузере

6. Меняем данные прямо в браузере

```
1 import marimo as mo
2 import pandas as pd
```

```
1 # Задание 1: Сравнение Marimo и Colab
```

r f i markdown

Задание 1: Сравнение Marimo и Colab

```
1 # Создаем DataFrame
2 df = pd.DataFrame({
3     "Пассажиры": ["Алена", "Иван", "Мария"],
4     "Выжил": [1, 0, 1],
5     "Класс": [1, 3, 2]
6 })
```

```
1 # Отображаем таблицу
2 df
```

PYTHON

MARKDOWN

SQL

GENERATE WITH AI

Рисунок 4 – Было изначально

```
1 import marimo as mo
2 import pandas as pd
```

```
1 # Задание 1: Сравнение Marimo и Colab
```

[r](#) [O](#) [f](#) [O](#) [i](#) | [markdown](#)

Задание 1: Сравнение Marimo и Colab

```
1 # Создаем DataFrame
2 df = pd.DataFrame({
3     "Пассажир": ["Василий", "Иван", "Мария"],
4     "Выжил": [1, 0, 1],
5     "Класс": [1, 3, 2]
6 })
```

```
1 # Отображаем таблицу
2 df
```

[Visualize](#) [Explore](#) [Export](#)

Пассажир object	Выжил int64	Класс int64	
Василий	1	1	
Иван	0	3	
Мария	1	2	

3 rows, 3 columns

No selection

[PYTHON](#)[MARKDOWN](#)[SQL](#)[GENERATE WITH AI](#)

Рисунок 5 – Изменение имени пассажира с Алена на Василий

```

marimo_test.py > ...
1  import marimo
2
3  __generated_with__ = "0.23.3"
4  app = marimo.App()
5
6
7  @app.cell
8  def _():
9      import marimo as mo
10     import pandas as pd
11
12     return mo, pd
13
14
15  @app.cell
16  def _(mo):
17      mo.md("""
18      # Задание 1: Сравнение Marimo и Colab
19      """)
20      return
21
22
23  @app.cell
24  def _(pd):
25      # Создаем DataFrame
26      df = pd.DataFrame({
27          "Пассажир": ["Василий", "Иван", "Мария"],
28          "Выжил": [1, 0, 1],
29          "Класс": [1, 3, 2]
30      })
31      return (df,)
32
33
34  @app.cell
35  def _(df):
36      # Отображаем таблицу
37      df
38      return
39
40
41  if __name__ == "__main__":
42      app.run()

```

Рисунок 6 – Автоматическое изменение имени пассажира в файле

Вывод

Традиционные ноутбуки (Colab) имеют три ключевых ограничения для MLOps.

Первое – проблема скрытого состояния. В Colab ячейки можно запускать в любом порядке. Например, можно изменить переменную в первой ячейке, не перезапустив последнюю, и получить неверный результат. Marimo решает это через реактивность – любое изменение в коде мгновенно обновляет все зависимые ячейки, что гарантирует воспроизводимость эксперимента.

Второе – формат файлов. Файл Colab (.ipynb) – это запутанный JSON с метаданными и служебной информацией. При сравнении двух версий такого файла в Git трудно понять, что именно изменилось в коде. Marimo сохраняется в обычный .py файл – можно видеть только свой код, без лишнего “мусора”. Это критически важно для командной разработки.

Третье – готовность к продакшену. Код из Marimo – это уже готовый Python-скрипт. Его можно сразу запустить на сервере, обернуть в Docker или встроить в автоматический пайплайн без переписывания. Colab для этого не предназначен.

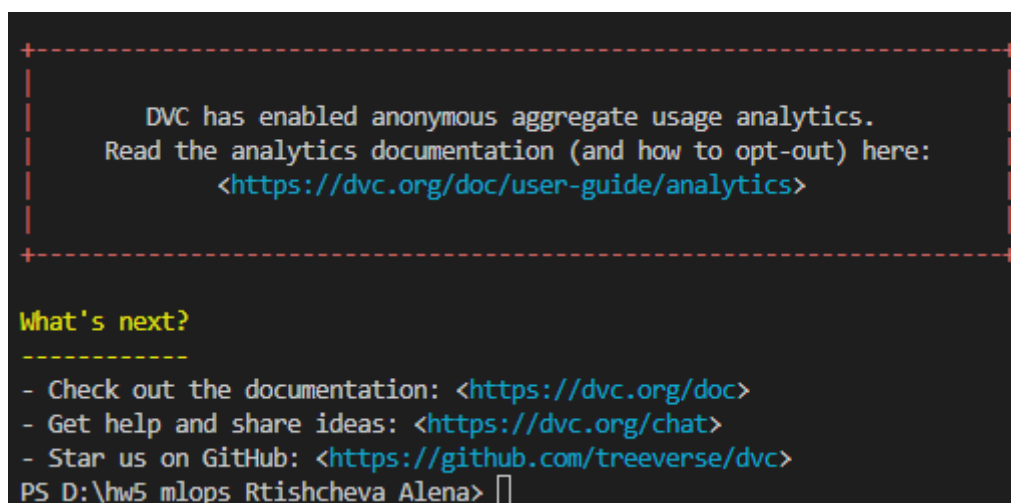
Таким образом, Colab хорош для быстрого прототипирования, но для промышленных ML-проектов Marimo – более правильный выбор.

2. Обеспечение воспроизводимости эксперимента, используя DVC/MLflow

1. Инициализируем DVC

```
dvc init --subdir
```

Появилась директория .dvc



```
+-----+
|       |
| DVC has enabled anonymous aggregate usage analytics. |
| Read the analytics documentation (and how to opt-out) here: |
| <https://dvc.org/doc/user-guide/analytics> |
|       |
+-----+

What's next?
-----
- Check out the documentation: <https://dvc.org/doc>
- Get help and share ideas: <https://dvc.org/chat>
- Star us on GitHub: <https://github.com/treverse/dvc>
PS D:\hw5_mlops_Rtishcheva_Alena> |
```

Рисунок 7 – Инициализация DVC

2. Создаем папку для скриптов

```
mkdir src
```

3. Создаем структуру данных

```
mkdir data
```

```
mkdir data/raw
```

```
mkdir data/processed
```

4. Создаем файлы в корне проекта

```
new-item params.yaml, requirements.txt, README.md
```

5. Для проекта выбран датасет Titanic. В связи с сетевыми ограничениями при автоматической загрузке (ошибка getaddrinfo failed), данные были загружены вручную с официального репозитория и помещены в папку проекта.

6. Согласно требованиям задания, файл был переименован в data.csv и размещен по пути data/raw/data.csv. Это обеспечивает единообразие пайплайна: скрипты всегда обращаются к файлу с фиксированным именем, независимо от его происхождения.

7. Добавляем файл в DVC.

```
dvc add data/raw/data.csv
```

```
(venv) PS D:\hw5_mlops_Rtishcheva_Alena> dvc add data/raw/data.csv
100% Adding...|
To track the changes with git, run:
    git add 'data\raw\data.csv.dvc' 'data\raw\.gitignore'
To enable auto staging, run:
    dvc config core.autostage true
(venv) PS D:\hw5_mlops_Rtishcheva_Alena>
```

Рисунок 8 – Появился файл data.csv.dvc

8. Создайте файлы пайплайна

```
src > prepare.py > ...
1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  import yaml
4  import os
5
6  # Читаем параметры из params.yaml
7  with open("params.yaml", "r") as f:
8      config = yaml.safe_load(f)
9
10 # Читаем данные, за которыми следит DVC
11 df = pd.read_csv("data/raw/data.csv")
12 df = df.select_dtypes(include=['number']).dropna() # Оставляем только числа
13
14 # Делим на train и test
15 train, test = train_test_split(df, test_size=config['prepare']['split_ratio'], random_state=42)
16
17 # Сохраняем результат
18 os.makedirs("data/processed", exist_ok=True)
19 train.to_csv("data/processed/train.csv", index=False)
20 test.to_csv("data/processed/test.csv", index=False)
```

Рисунок 9 – Файл src/prepare.py (разделение данных)

```

src > train.py > ...
1  import pandas as pd
2  from sklearn.linear_model import LogisticRegression
3  import pickle
4  import mlflow
5  import yaml
6
7  with open("params.yaml", "r") as f:
8      config = yaml.safe_load(f)
9
10 train = pd.read_csv("data/processed/train.csv")
11 X = train.drop("Survived", axis=1, errors='ignore')
12 y = train["Survived"]
13
14 # Настройка локального лога
15 mlflow.set_tracking_uri("sqlite:///mlflow.db")
16
17 with mlflow.start_run():
18     model = LogisticRegression(max_iter=config['train']['max_iter'])
19     model.fit(X, y)
20
21     # Записываем результаты в MLflow
22     mlflow.log_param("max_iter", config['train']['max_iter'])
23     mlflow.log_metric("accuracy", model.score(X, y))
24
25     # Сохраняем модель
26     with open("model.pkl", "wb") as f:
27         pickle.dump(model, f)
28     mlflow.log_artifact("model.pkl")

```

Рисунок 10 – Файл src/train.py (обучение + MLflow)

8. Запускаем стадии пайплайна

Первая стадия подготовки

dvc stage add -n prepare -d src/prepare.py -d data/raw/data.csv -o data/processed python src/prepare.py

Вторая стадия - обучение

dvc stage add -n train -d src/train.py -d data/processed -o model.pkl python src/train.py

При этом автоматически создался файл dvc.yaml

```
! dvc.yaml
1  stages:
2    prepare:
3      cmd: python src/prepare.py
4      deps:
5        - data/raw/data.csv
6        - src/prepare.py
7      outs:
8        - data/processed
9    train:
10     cmd: python src/train.py
11     deps:
12       - data/processed
13       - src/train.py
14     outs:
15       - model.pkl
16
```

Рисунок 11 – Автоматически созданный файл dvc.yaml

9. Корректируем params.yaml

```
! params.yaml
1  prepare:
2    split_ratio: 0.2
3  train:
4    max_iter: 100
5
```

Рисунок 12 – params.yaml

10. Проверяем работу пайплайна dvc repro

```
(venv) PS D:\hw5_mlops_Rtishcheva_Alena> dvc repro
'data\raw\data.csv.dvc' didn't change, skipping
Stage 'prepare' didn't change, skipping
Running stage 'train':
> python src/train.py
2026/04/26 21:13:48 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/04/26 21:13:49 INFO mlflow.store.db.utils: Updating database tables
2026/04/26 21:14:28 WARNING mlflow.utils.git_utils: Failed to import Git (the Git executable is probably not on your PATH), so Git SHA is not available. Error: Failed to initialize: Bad git executable.
The git executable must be specified in one of the following ways:
- be included in your $PATH
- be set via $GIT_PYTHON_GIT_EXECUTABLE
- explicitly set via git.refresh(<full-path-to-git-executable>)

All git commands will error until this is rectified.

This initial message can be silenced or aggravated in the future by setting the
$GIT_PYTHON_REFRESH environment variable. Use one of the following values:
- quiet|q|silence|s|silent|none|n|0: for no message or exception
- warn|w|warning|log|l|1: for a warning message (logging level CRITICAL, displayed by default)
- error|e|exception|raise|r|2: for a raised exception

Example:
export GIT_PYTHON_REFRESH=quiet

D:\hw5_mlops_Rtishcheva_Alena\venv\Lib\site-packages\sklearn\linear_model\_logistic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
Updating lock file 'dvc.lock'

To track the changes with git, run:

    git add dvc.lock

To enable auto staging, run:

    dvc config core.autostage true
Use 'dvc push' to send your updates to remote storage.
(venv) PS D:\hw5_mlops_Rtishcheva_Alena>
```

Рисунок 13 – Обработка паплайна

11. После обработки паплайна появились новые файлы: dvc.lock, model.pkl, mlflow.db.

Выводы

В рамках данного этапа была построена архитектура воспроизводимого машинного обучения, решающая проблему «непрозрачных» экспериментов.

Реализовано разделение хранения кода и данных. Исходный датасет был вынесен под контроль DVC, что позволило исключить хранение тяжелых бинарных файлов в Git. Теперь Git отслеживает только метафайлы (.dvc), гарантируя связь конкретной версии кода с версией данных.

С помощью файла dvc.yaml был сконфигурирован конвейер, состоящий из стадий prepare (подготовка и сплит) и train (обучение). Это обеспечило автоматизацию: команда dvc repro самостоятельно отслеживает изменения и перезапускает только необходимые этапы.

Все ключевые параметры вынесены в файл params.yaml. Это позволяет изменять конфигурацию эксперимента без вмешательства в исходный код скриптов.

В проект интегрирована система трекинга MLflow. Все параметры и метрики качества (ассурасу) автоматически записываются в локальную базу данных (mlflow.db). Это обеспечивает сохранность результатов каждого эксперимента и позволяет проводить аудит обученных моделей.

3. Создание хранилища признаков (Feature Store)

1. Инициализируем Feast
feast init feature_repo

```
(venv) PS D:\hw5_mlops_Rtishcheva_Alena> feast init feature_rep
C:\Users\Alena\AppData\Local\Programs\Python\Python313\Lib\site-packages\asttokens\ut
om 'astroid.nodes' instead
from astroid import Const

Creating a new Feast repository in D:\hw5_mlops_Rtishcheva_Alena\feature_rep.

(venv) PS D:\hw5_mlops_Rtishcheva_Alena> █
```

Рисунок 14 – Инициализация Feast

2. Появилась папка feature_repo
3. Изменяем файл feature_store.yaml

```
feature_repo > feature_repo > ! feature_store.yaml
1  project: feature_repo
2  # Реестр метаданных
3  registry: data/registry.db
4  # Провайдер остается локальным для управления, но хранилище – внешним
5  provider: local
6
7  # Офлайн-хранилище (откуда берем данные для обучения)
8  offline_store:
9    type: file
10
11 # Онлайн-хранилище (куда отдаем признаки на эндпоинт)
12 online_store:
13   type: postgres
14   host: 127.0.0.1
15   port: 5433
16   database: feast_online
17   user: postgres
18   password: password
19   sslmode: disable
20
21 entity_key_serialization_version: 3
22 █
```

Рисунок 15 – Файл feature_store.yaml

4. Установим недостающую библиотеку
pip install "feast[postgres]"
5. Запускаем Docker Desktop
6. Запуск PostgreSQL через Docker
docker run --name feast-postgres -e POSTGRES_PASSWORD=password -e POSTGRES_DB=feast_online -p 5432:5432 -d postgres

```
(venv) PS D:\hw5_mlops_Rtishcheva_Alena\feature_repo\feature_repo> docker run --name feast-postgres -e POSTGRES_PASSWORD=password -e POSTGRES_DB=feast_online -p 5432:5432 -d postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
e57497b6cf23: Download complete
e57497b6cf23: Pull complete
b7036600a44b: Pull complete
e9cad8a9e695: Pull complete
6fadfe1078a3: Pull complete
4e2103318c3c: Pull complete
3531af2bc2a9: Pull complete
56b9e522a2ac: Pull complete
13f7f91984c8: Pull complete
2428a49310a9: Pull complete
85f6ddcd85b0: Pull complete
c43308a19dbe: Pull complete
c3f159f3b349: Pull complete
6e695e05adc6: Download complete
631e25198ba2: Download complete
Digest: sha256:78481659c47e862334611ccdaf7c369c986b3046da9857112f3b309114a65fb4
Status: Downloaded newer image for postgres:latest
9eb02da5cb260d675406e9518da033c5cec2de3e001b139a0abe61d4896084a3
(venv) PS D:\hw5_mlops_Rtishcheva_Alena\feature_repo\feature_repo>
```

Рисунок 16 – Docker скачал образ и запустил сервер PostgreSQL

☐ ● feast-postgres b6cb74228e55 [postgres](#) [5433:5432](#) [↗](#)

Рисунок 17 – Создан и запущен контейнер с базой данных

7. Запуск регистрации признаков

```
cd feature_repo
cd feature_repo
feast apply
```

```
(venv) PS D:\hw5_mlops_Rtishcheva_Alena\feature_repo\feature_repo> feast apply
D:\hw5_mlops_Rtishcheva_Alena\feature_repo\feature_repo\feature_definitions.py:27: DeprecationWarning: Entity value_type will
iver'.
    driver = Entity(name="driver", join_keys=["driver_id"])
Applying changes for project feature_repo
D:\hw5_mlops_Rtishcheva_Alena\venv\Lib\site-packages\feast\feature_store.py:690: RuntimeWarning: On demand feature view is an
le well for offline retrieval
    warnings.warn(
Deploying infrastructure for driver_hourly_stats
Deploying infrastructure for driver_hourly_stats_fresh
(venv) PS D:\hw5_mlops_Rtishcheva_Alena\feature_repo\feature_repo>
```

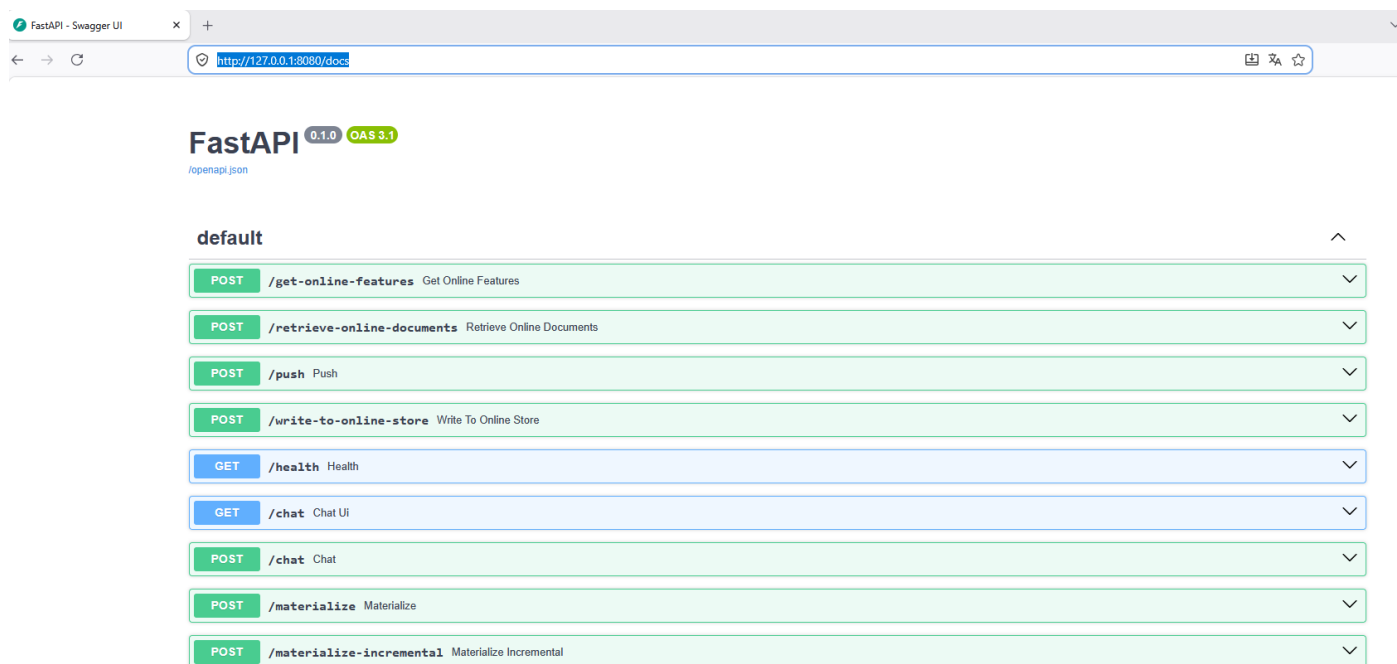
Рисунок 18 – Хранилище признаков Feast настроено

8. Запускаем эндпоинт

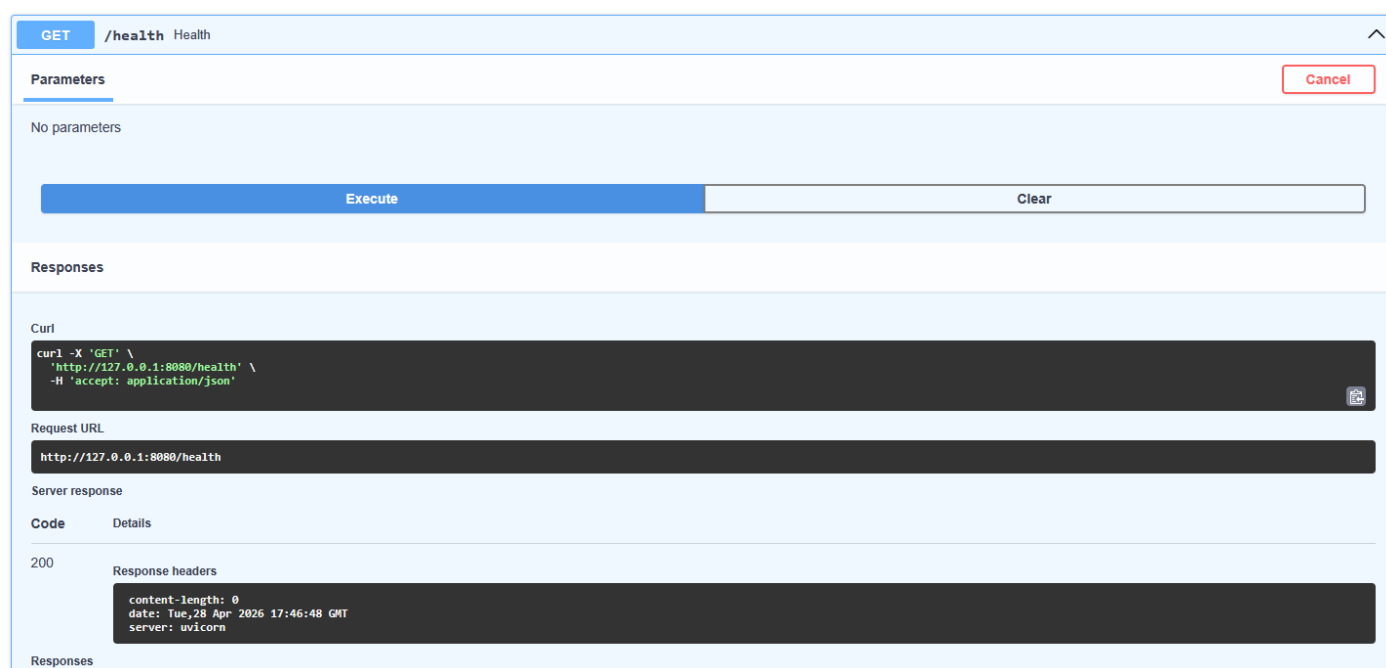
```
feast serve -p 8080
```

```
(venv) PS D:\hw5_mlops_Rtishcheva_Alena\feature_repo\feature_repo> feast serve -p 8080
INFO:fastapi:Auth type: AuthManagerType.NONE
D:\hw5_mlops_Rtishcheva_Alena\venv\Lib\site-packages\websockets\legacy\__init__.py:6: DeprecationWarning: websockets.legacy i
nstructions
    warnings.warn( # deprecated in 14.0 - 2024-11-09
D:\hw5_mlops_Rtishcheva_Alena\venv\Lib\site-packages\uvicorn\protocols\websockets\websockets_impl.py:17: DeprecationWarning:
    from websockets.server import WebSocketServerProtocol
INFO: Started server process [9136]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO: 127.0.0.1:49823 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:49824 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:49825 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:49825 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:49898 - "GET /health HTTP/1.1" 200 OK
```

Рисунок 19 – Запуск сервера



a)



б)

Рисунок 20 – Визуализация эндпоинта

Вывод:

В рамках данного этапа в архитектуру проекта был внедрен инструмент Feast, выполняющий роль централизованного хранилища признаков (Feature Store).

Для обеспечения масштабируемости системы был использован шаблон PostgreSQL. Файл конфигурации feature_store.yaml успешно прошел валидацию, подтвердив готовность системы к работе с внешними базами данных. В ходе работы был развернут и настроен Docker-контейнер с PostgreSQL (feast-postgres), что позволило протестировать онлайн-хранилище в условиях, максимально приближенных к реальному продакшену. С помощью команды feast apply была успешно развернута инфраструктура признаков. Успешно запущен сервис feast serve, предоставляющий

рабочий эндпоинт на порту 8080. Это позволяет внешним сервисам получать признаки (features) в реальном времени через стандартизированный API.

4. Обоснование готовности ML-системы к развертыванию в продуктивном окружении

Промышленная ML-система (в отличие от обычного скрипта) включает в себя:

- Сбор и версионирование данных (DVC): Отслеживание изменений в датасетах.
- Feature Store (Feast): Централизованное хранилище признаков для обучения и инференса.
- Эксперимент-менеджмент (MLflow): Логирование каждой итерации обучения.
- Model Registry: Хранилище версий моделей, готовых к эксплуатации.
- Serving Layer (API): Сервис (например, FastAPI), который принимает данные и отдает

предсказания.

- Мониторинг: Отслеживание качества модели и «дрифта» (сдвига) данных в реальном времени.

- CI/CD пайплайны: Автоматические тесты кода и моделей перед выпуском.

Для текущего проекта (детекция и размытие лиц):

- DVC и MLflow – необходимы, так как работа ведется с изображениями и видеоданными и важно знать, на каких данных и с какими параметрами обучена модель.

- Feature Store – полезен, если мы будем использовать дополнительные признаки (например, метаданные о времени или месте съемки) для улучшения точности.

- Мониторинг и API - критичны, если мы хотим встроить размытие лиц в реальный видеопоток (например, на камерах видеонаблюдения), чтобы вовремя заметить, если модель перестала узнавать лица при плохом освещении.

Система не готова к полноценному развертыванию.

Хотя наличие репозитория и блокнота – хороший старт, этого недостаточно по следующим причинам:

1. Проблема «скрытого состояния»: Блокноты (особенно Colab) зависят от порядка запуска ячеек и временной среды. Это нарушает принцип воспроизводимости.
2. Отсутствие инференс-сервиса: Блокнот нельзя вызвать как API-функцию из другого приложения.
3. Нет гарантии стабильности: Блокнот не обеспечивает автоматическую проверку ошибок (тестирование) и не предназначен для высоких нагрузок.
4. Отсутствие мониторинга: Мы не узнаем, если в реальных условиях точность модели упадет.

Таким образом, текущий проект является качественным MVP (минимально жизнеспособным продуктом). Благодаря DVC, MLflow и Feast был заложен фундамент, но для «продакшена» систему необходимо упаковать в микросервис (через созданный Dockerfile) и настроить мониторинг.

5. Схема ML-системы для анонимизации лиц

Анализ текущего решения

Предоставленный код является синхронным и однопоточным. Он обрабатывает кадры видео один за другим в едином цикле. Это неэффективно для больших данных и создает риски задержек при обработке конфиденциальной информации.

Схематическое описание (Architecture)

Для промышленного использования предлагается архитектура, разделенная на независимые слои:

1. Слой данных (Storage Layer): Видеофайлы и веса моделей классификатора (Haar Cascades) версионизируются с помощью DVC.
2. Слой параллельных вычислений (Processing Layer):

- Видео разбивается на независимые фрагменты (чанги).
- Несколько Worker-узлов (Docker-контейнеры) одновременно обрабатывают фрагменты, используя библиотеку OpenCV.

3. Слой признаков (Feature Layer): С помощью Feast подтягиваются актуальные параметры анонимизации (например, pixel_size для разного качества видео).

4. Слой мониторинга (Tracking Layer): MLflow фиксирует метрики производительности и метаданные каждого запуска.

Данная схема позволяет масштабировать обработку видео пропорционально количеству доступных вычислительных ресурсов. Параллелизм на уровне данных сокращает время анонимизации в несколько раз, что критически важно для соблюдения юридических норм защиты персональных данных.

Вывод

Реализован полный цикл MLOps-конвейера: от версионирования данных в DVC до развертывания хранилища признаков в Feast. Работа с MLflow позволила структурировать процесс обучения, превратив хаотичные запуски в прозрачный журнал экспериментов с сохранением всех артефактов. Наибольшую трудность вызвала настройка PostgreSQL в Docker для Feature Store, что дало ценный практический опыт отладки сетевых соединений в изолированных средах. Переход от блокнотов Colab к Marimo и промышленным пайплайнам показал преимущество инженерного подхода в обеспечении воспроизводимости и стабильности ML-систем. Итоговая архитектура анонимизации видео продемонстрировала готовность системы к масштабированию.